

Báo cáo dự án

tmath AI Diagnostic & Admin Dashboard Service

Cấu trúc dự án – Tiến độ – Phân công phát triển

Dũng, Khải

Ngày 08 tháng 09 năm 2026

Tài liệu tham chiếu: docs/SOFTWARE_DESIGN_DOCUMENT.md (SDD v3.1.0), docs/FEATURE_1_TAG_ANALYTICS_SPEC.md (FEAT-01), docs/WORK_SPLIT_PLAN.md.

Contents

1	Cấu trúc dự án	2
1.1	Tổng quan hệ thống	2
1.2	Cây thư mục	2
2	Tiến độ dự án	3
2.1	Đã hoàn thành	3
2.2	Điểm gấn cứng cần thay bằng dữ liệu thật	3
2.3	Tính năng còn thiếu theo SDD	4
2.4	Nợ kỹ thuật	4
3	Phân công phát triển	4
3.1	Milestone 0 chung: dựng môi trường, chạy thật (1–2 ngày)	4
3.2	Dũng — Domain Học sinh và AI cá nhân hóa	4
3.3	Khải — Domain Giáo viên, Admin và Pipeline AI	5
3.4	Quy tắc phối hợp	5
3.5	Lộ trình tổng hợp	5

1. Cấu trúc dự án

1.1 Tổng quan hệ thống

Hệ thống là một microservice phục vụ nền tảng tmath Online Judge (DMOJ), gồm hai phần chính: backend API phân tích học lực bằng AI và dashboard web cho học sinh, giáo viên, admin.

- **Backend:** Python 3.12, FastAPI, SQLAlchemy Async 2.0, aiomysql, kết nối trực tiếp MySQL 8.0 gốc của tmath (backup 10.1 GB, 200 file SQL).
- **AI Orchestration:** Instructor Framework (structured output Pydantic) + LiteLLM Adapter, chạy trên Ollama local (Qwen2.5-Coder) hoặc Gemini Cloud API.
- **Frontend:** React 18 + TypeScript + Vite + TailwindCSS + Recharts, component UI kiểu shadcn.
- **Hạ tầng:** Docker Compose 6 service (MySQL, Redis, Ollama, Open WebUI, backend, frontend), kèm cấu hình GPU NVIDIA CUDA / AMD ROCm.

1.2 Cây thư mục

```
tmathcoding-ai/
├── docker-compose.yml           # Stack: db, redis, ollama, open-webui, backend, frontend
├── docker-compose.gpu.yml      # Override NVIDIA CUDA
├── docker-compose.rocm.yml     # Override AMD ROCm
├── docs/
│   ├── SOFTWARE_DESIGN_DOCUMENT.md    # SDD v3.1.0
│   ├── FEATURE_1_TAG_ANALYTICS_SPEC.md # Đặc tả FEAT-01
│   └── WORK_SPLIT_PLAN.md              # Kế hoạch phân công
├── backup/                     # 200 file SQL (10.1 GB)
├── scripts/
│   ├── seed_mysql.py           # Import backup vào MySQL
│   ├── pull_model.py          # Nạp model AI vào Ollama
│   └── test_llm.py             # Chat CLI thử LLM
├── backend/
│   ├── requirements.txt
│   └── app/
│       ├── main.py             # FastAPI app + CORS + /health
│       ├── core/
│       │   ├── config.py       # Pydantic Settings, đọc .env
│       │   ├── database.py     # Async engine + session
│       │   ├── auth.py         # Dev auth qua header X-User-ID
│       │   └── llm_adapter.py  # Instructor + LiteLLM adapter
│       ├── models/dmoj.py      # 11 bảng ánh xạ DB thật
│       ├── schemas/            # Pydantic: analytics, student, ai, teacher
│       ├── services/
│       │   ├── tag_analytics_service.py # FEAT-01: metric theo tag
│       │   ├── tag_ai_service.py       # AI Advisor hằng ngày
│       │   ├── skill_tree_service.py   # F1.1 Cây kỹ năng
│       │   ├── code_doctor_service.py  # F1.2 Code Doctor
│       │   └── heatmap_service.py      # F2.1/F2.2 Heatmap + chi tiết
│       └── api/v1/endpoints/          # student.py, teacher.py, admin.py
└── frontend/
```

```

├─ vite.config.ts           # Proxy /api -> backend
├─ src/
│   ├─ App.tsx             # App shell + tìm kiếm toàn cục
│   ├─ services/api.ts     # Axios API layer
│   ├─ types/index.ts      # Kiểu dữ liệu khớp backend
│   ├─ pages/              # StudentDashboard, TeacherDashboard
│   └─ components/         # SkillTree, BloomRadar, TagCompletionGrid,
                           # AIAdvisorCard, CodeDoctorModal, ClassHeatmap...

```

2. Tiến độ dự án

Mã nguồn hiện tại đã đi xa hơn mức khởi tạo: các tính năng phía Học sinh và Giáo viên đều có logic thật ở cả backend lẫn frontend. Nguyên nhân hệ thống chưa chạy được nằm ở ba nhóm: môi trường chưa dựng, một số điểm dữ liệu còn gán cứng, và hai tính năng trong SDD chưa triển khai.

2.1 Đã hoàn thành

Thành phần	Trạng thái	Ghi chú
Backend core (config, async MySQL, LLM Adapter Instructor + LiteLLM)	Xong	backend/app/core/
11 model ánh xạ DB thật tmath (DMOJ)	Xong	models/dmoj.py
FEAT-01: Tag Analytics + AI Advisor hằng ngày	Xong logic	Có fallback khi LLM lỗi
F1.1 Cây kỹ năng 99 node	Xong logic	Riêng Bloom Radar còn số cứng
F1.2 Code Doctor (Socratic)	Xong logic	Có fallback theo loại lỗi
F2.1 Heatmap 2D + cảnh báo STUCK / GAP / INACTIVE	Xong logic	Dữ liệu Bloom thật
F2.2 Tìm kiếm + chi tiết học sinh	Xong logic	Có modal chi tiết
Frontend: app shell, sidebar, tìm kiếm toàn cục, 2 dashboard	Xong	Biên dịch TypeScript sạch
Hạ tầng: Docker Compose, script seed DB, nạp model	Xong	200 file SQL backup

2.2 Điểm gán cứng cần thay bằng dữ liệu thật

- Bloom Radar** của Cây kỹ năng trả về điểm mô phỏng theo số chủ đề (hàm trong `skill_tree_service.py`), không phải điểm Bloom thật từ bảng `judge_problemgrou`.
- Code Doctor** ở giao diện học sinh gọi cứng mã bài nộp 3447100, chưa có cơ chế chọn bài nộp lỗi thật.
- Admin auto-tag** trả về số liệu cứng (16.491 bài / 4.520 đã tag / 27,4%), chưa có pipeline thật.

2.3 Tính năng còn thiếu theo SDD

- **F3.1 Pipeline Auto-Tagging** (Code-Informed Classifier): mới có schema `AutoTagResult`, chưa có worker trích đặc trưng từ mã nguồn AC, chưa lưu kết quả vào CSDL.
- **F1.3 Chế độ tự học** (Independent Learning Mode): chưa có code.

2.4 Nợ kỹ thuật

- Redis và Celery khai báo trong requirements và Docker Compose nhưng chưa được sử dụng; cache nhận xét AI đang lưu trong bộ nhớ của tiến trình, mất khi khởi động lại.
- Các endpoint giáo viên chưa kiểm tra quyền: mọi giá trị `org_id / teacher_id` đều đọc được dữ liệu.
- Chưa có bộ kiểm thử tự động (pytest).
- Môi trường chạy thật chưa dựng: dependencies Python chưa cài trên máy, Docker stack chưa chạy, DB chưa seed, model Ollama chưa nạp.

3. Phân công phát triển

Hai thành viên **Dũng** và **Khải** đều là fullstack. Chia việc theo domain dọc: mỗi người chịu trách nhiệm toàn bộ backend, frontend và kiểm thử của domain mình để hạn chế xung đột khi gộp mã nguồn.

3.1 Milestone 0 chung: dựng môi trường, chạy thật (1–2 ngày)

Mã	Công việc	Phụ trách
M0.1	Khởi chạy Docker Compose, seed 10.1 GB backup, nạp model Ollama theo README	Cả hai (mỗi người một bộ môi trường)
M0.2	Sửa <code>docker-compose.yml</code> và các script seed / nạp model nếu lỗi; cập nhật README	Khải , Dũng review
M0.3	Hướng dẫn chạy dev nhẹ không cần full Docker: container db + redis + ollama, backend chạy uvicorn local	Dũng , Khải review
M0.4	Smoke test: <code>/health</code> , 4 API học sinh, 5 API giáo viên, 2 dashboard hiển thị đủ dữ liệu	Cả hai

3.2 Dũng — Domain Học sinh và AI cá nhân hóa

Mã	Công việc	Tuần
A1	Bloom Radar thật: tính điểm Bloom từ <code>judge_problem.group_id</code> theo <code>BLOOM_GROUP_MAP</code> (4=A, 5=B, 6=C, 7=D, 8=E, 13=F); sửa <code>skill_tree_service.py</code> , <code>schemas/student.py</code> , <code>BloomRadar.tsx</code>	1
A2	Code Doctor thật: thêm API liệt kê bài nộp lỗi gần đây (WA / TLE / RTE, join <code>judge_submissionsource</code>); thay nút mock bằng danh sách chọn bài nộp, có trạng thái rõ ràng	2

Mã	Công việc	Tuần
A3	Cache Redis cho AI Advisor: thay bộ nhớ trong tiến trình bằng key <code>ai_commentary:{user_id}:{ngày}</code> ; <code>force_refresh</code> vẫn xóa cache	3
A4	F1.3 Chế độ tự học: gợi ý lộ trình dựa trên tag yếu nhất từ FEAT-01 (trạng thái <code>NEEDS_IMPROVEMENT / UNATTEMPTED</code>); 1 endpoint mới + 1 section mới trong StudentDashboard	3–4
A5	Chất lượng: pytest cho 3 service phía học sinh; audit giao diện theo ANTIS-LOP.md	4

3.3 Khái — Domain Giáo viên, Admin và Pipeline AI

Mã	Công việc	Tuần
B1	F3.1 Pipeline Auto-Tagging thật: tạo bảng <code>judge_problem_ai_tag</code> lưu kết quả; trích đặc trưng từ mã nguồn AC (<code>dfs/bfs/queue</code> , <code>dp</code> , <code>segment_tree</code> , <code>gcd</code> , sàng số, giới hạn N) theo SDD mục 4.3 và 6; gọi LLM qua schema <code>AutoTagResult</code> ; chạy batch bằng <code>BackgroundTasks</code> ; status đọc từ DB	1–2
B2	Trang Admin: thêm tab mới trong AppSidebar với thanh tiến độ, nút chạy batch, bảng 10 bài được tag gần nhất	2
B3	Phân quyền giáo viên: kiểm tra quyền quản lý lớp từ <code>judge_organization_admins</code> (hoặc <code>super admin</code>) trước khi trả dữ liệu; gom vào dependency riêng để về sau cấm JWT theo SDD mục 7	2–3
B4	Chất lượng và hạ tầng: pytest cho heatmap (điểm Bloom, 3 loại cảnh báo) và endpoint giáo viên; healthcheck cho service backend trong Docker Compose	3

3.4 Quy tắc phối hợp

- Hợp đồng API:** `app/schemas/*.py` và `frontend/src/types/index.ts` phải đồng bộ; ai đổi một phía phải đổi cả hai phía trong cùng commit và nêu trong pull request.
- Ranh giới file:** Dùng sở hữu các file phía student; Khái sở hữu teacher, admin, heatmap, auto-tag. File chung (`router.py`, `App.tsx`, `AppSidebar.tsx`, `types/index.ts`) chỉ được thêm, không sửa phần của người kia.
- Nhánh và review:** `feat/student-*` (Dũng), `feat/teacher-admin-*` (Khải), `infra/*` (Khải ở M0); mọi pull request vào main cần review của người còn lại.
- Thiết kế:** mọi thay đổi giao diện theo `DESIGN.md`, qua audit `ANTISLOP.md`; văn bản giao diện tiếng Việt ngắn gọn.
- Demo hàng tuần:** mỗi người demo domain của mình trên dữ liệu thật; lỗi ngoài domain ghi issue cho đúng người phụ trách.

3.5 Lộ trình tổng hợp

Tuần	Dũng	Khả
1	M0 + A1 (Bloom Radar thật)	M0 + B1 (Auto-Tagging pipeline thật)
2	A2 (Code Doctor thật)	B1 hoàn tất + B2 (trang Admin) + B3
3	A3 (Redis cache) + A4 (Chế độ tự học)	B3 hoàn tất + B4 (pytest, healthcheck)
4	A4 hoàn tất + A5 (kiểm thử, audit)	Xử lý nợ kỹ thuật, tối ưu hiệu năng